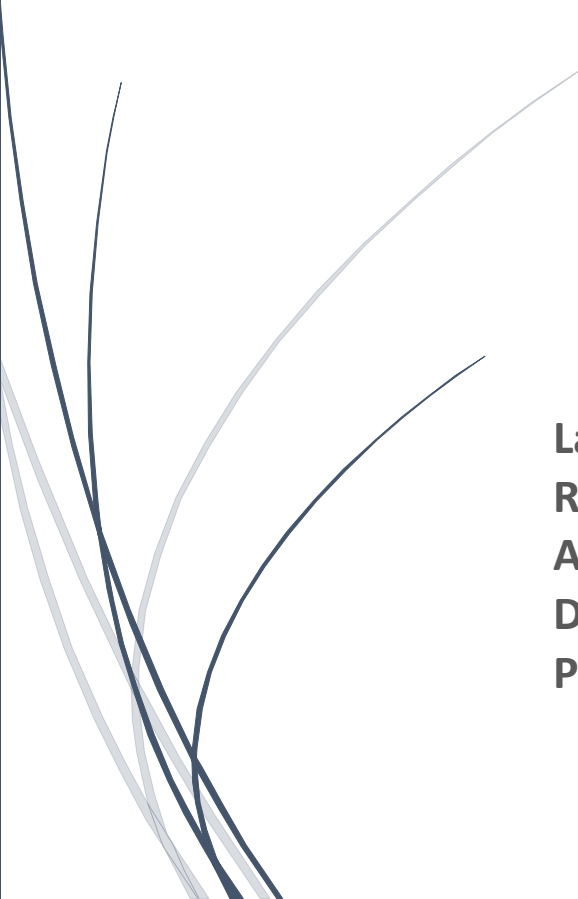


**SWT 22022 – Practical
for Internet Application
Development**

Department of Information
and Communication
Technology

Faculty of Technology



Lab sheet :18

Reg. Number: SEU/IS/20/ICT/084

Academic Year :2020/2021

Date: 2024.10.28

Practical No :18

Title: React Components

Aims:

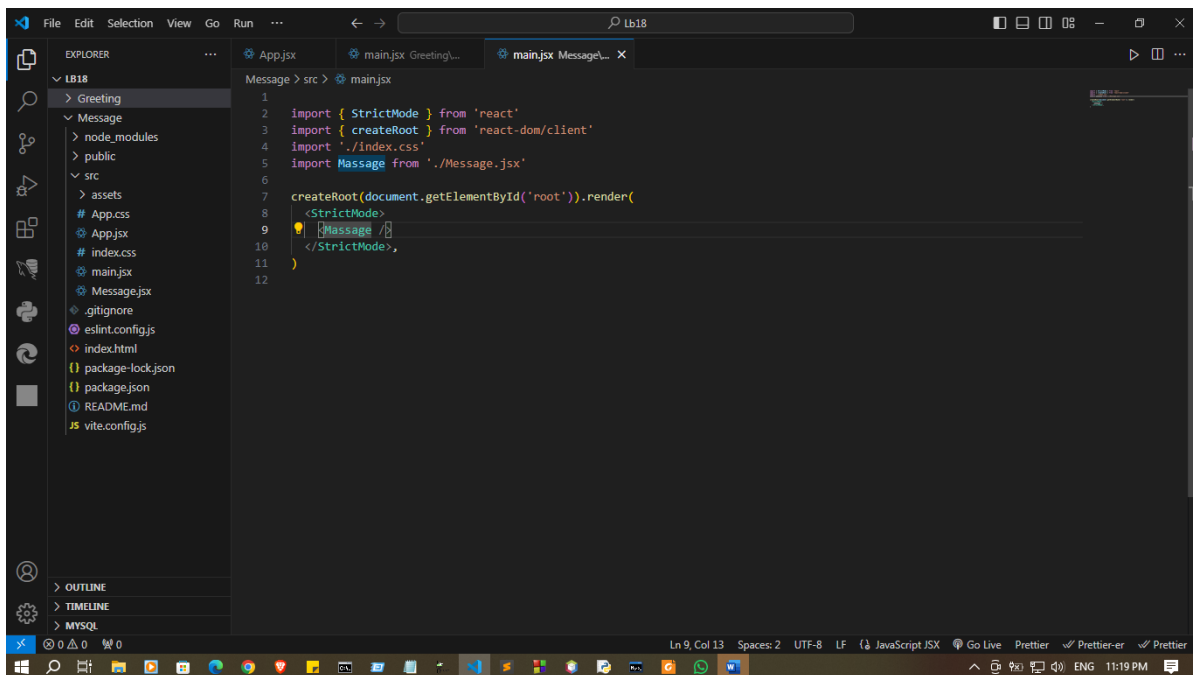
- Understand the concept of React components
- Create and use React components
- Pass data between components
- Update state changes

Tasks:

- Creating Components
- Using Components
- Image Inserting Passing props between components
- Updating state and re-rendering
- Building a simple form

Activities :

1. Create a basic component to display a message I. Create a new React project.
 - II. Create a new component named **Message**.
 - III. Define a functional component with a **render** method that returns a JSX element displaying the message.
 - IV. Import the component into your **App.js** file and render it.



The screenshot shows a VS Code editor window with a React project. The Explorer panel on the left shows the file structure: `lib18` (root) contains `Greeting`, `Message`, `node_modules`, `public`, `src` (containing `assets`, `App.css`, `App.jsx`, `index.css`, `main.jsx`, `Message.jsx`), `.gitignore`, `eslint.config.js`, `index.html`, `package-lock.json`, `package.json`, `README.md`, and `vite.config.js`. The main editor shows the `main.jsx` file with the following code:

```
1
2 import { StrictMode } from 'react'
3 import { createRoot } from 'react-dom/client'
4 import './index.css'
5 import Message from './Message.jsx'
6
7 createRoot(document.getElementById('root')).render(
8   <StrictMode>
9     <Message />
10  </StrictMode>,
11 )
12
```

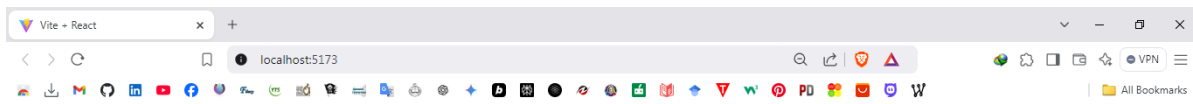
The status bar at the bottom indicates the current file is `main.jsx` at line 9, column 13, using UTF-8 encoding and LF line endings. The editor is configured with the JavaScript JSX language and has Prettier installed.

Message.jsx

```
function Message() {  
  return(  
    <>  
      <div>  
        <h1>Hello, this is a React Component!</h1>  
      </div>  
    </>  
  );  
}  
  
export default Message;
```

main.jsx

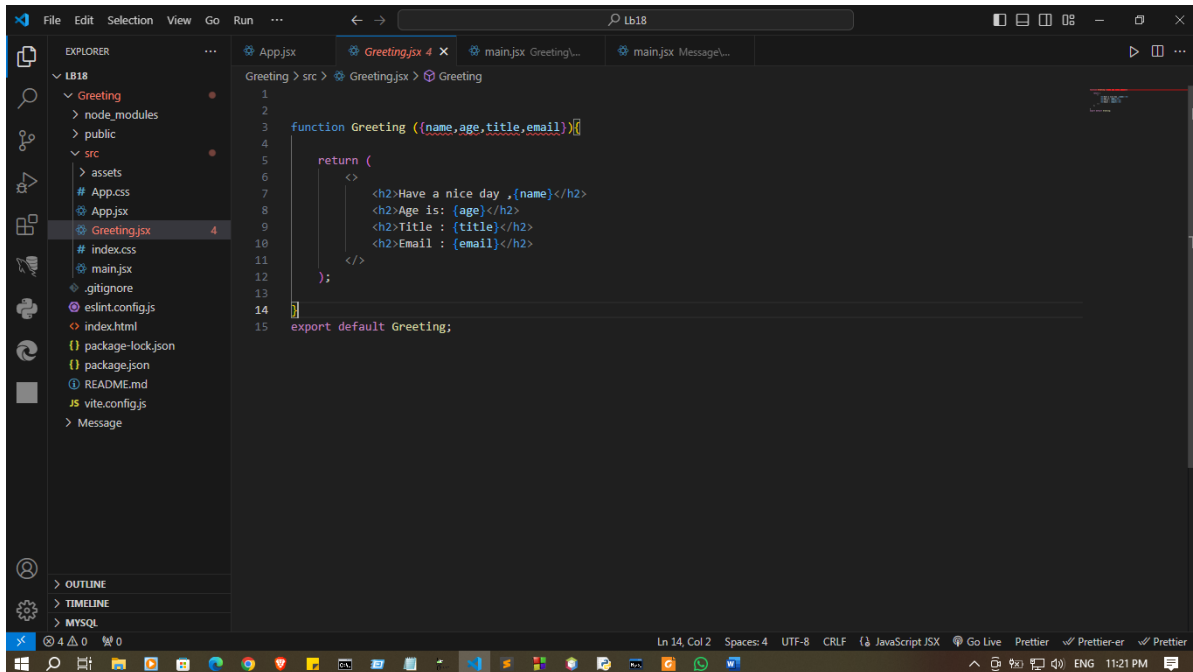
```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import './index.css'  
import Message from './Message.jsx'  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <Message />  
  </StrictMode>,  
)
```



Hello, this is a React Component!



2. Pass data from parent component to child component
 - I. Create new component named **Greeting**
 - II. Make the Greeting Component accept a **name** prop.
 - III. In **App.js**, pass the **name** prop to the **Greeting** component.
 - IV. Render the **name** prop within the **Greeting** component.



Greeting.jsx

```
function Greeting ({name,age,title,email}){  
  
    return (  
        <>  
            <h2>Have a nice day ,{name}</h2>  
            <h2>Age is: {age}</h2>  
            <h2>Title : {title}</h2>  
            <h2>Email : {email}</h2>  
        </>  
    );  
}  
export default Greeting;
```

App.jsx

```
import Greeting from './Greeting.jsx';  
import './App.css'  
  
function App() {  
    const name ="Kalhara";  
    const age =20;  
    const title ="Students";  
    const email="kalharap84@gmail.com";  
    return (  
        <>
```

```

    <div>
      <Greeting name={name} age={age} title={title} email={email}/>
    </div>
  </>
)
}

export default App

```

main.jsx

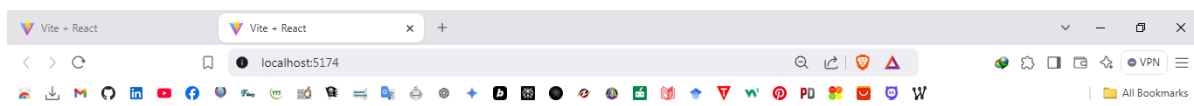
```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'

import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```



Have a nice day ,Kalhara

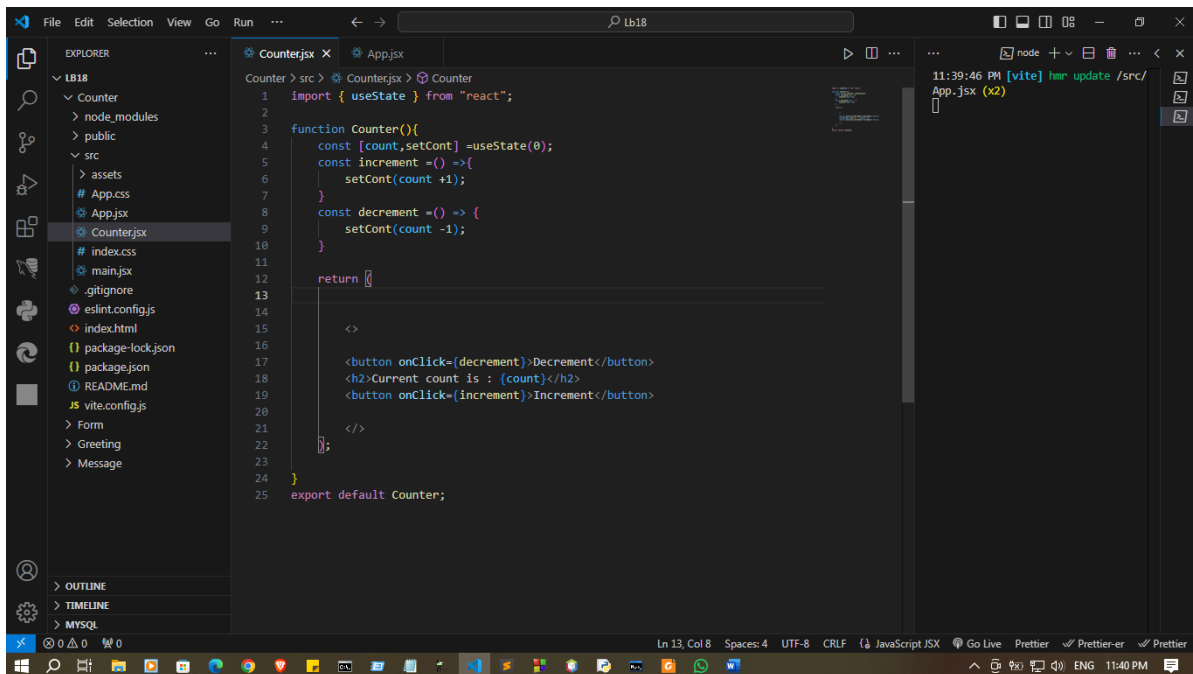
Age is: 20

Title : Students

Email : kalharap84@gmail.com



3. Update the state of a component and trigger a re-render
 - I. Create a new component name **Counter**.
 - II. Initialize the state with a **count** property.
 - III. Create functions to increment and decrement the **count**.
 - IV. Update the state using the **useState**.
 - V. Render the current **count** value.



Counter.jsx

```
import { useState } from "react";

function Counter(){
  const [count,setCont] =useState(0);
  const increment =() =>{
    setCont(count +1);
  }
  const decrement =() => {
    setCont(count -1);
  }

  return (

    <>

    <button onClick={decrement}>Decrement</button>
    <h2>Current count is : {count}</h2>
    <button onClick={increment}>Increment</button>

    </>

  );
}
export default Counter;
```

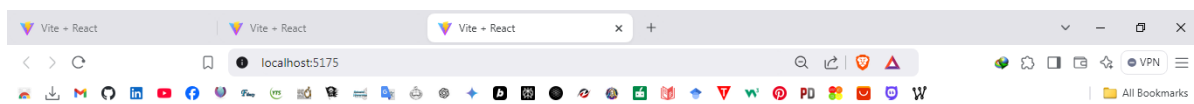
App.jsx

```
import './App.css'
import Counter from './Counter'
```

```
function App() {  
  
  return (  
    <>  
      <Counter />  
    </>  
  )  
}  
  
export default App
```

main.jsx

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import './index.css'  
import App from './App.jsx'  
  
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```



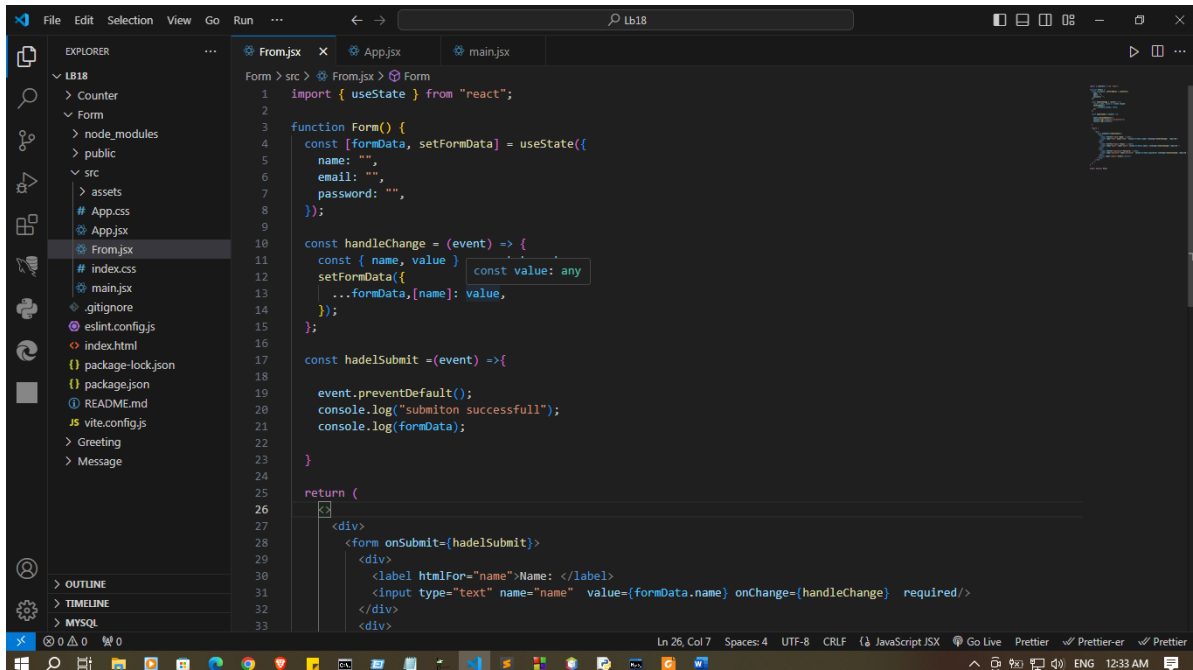
Decrement

Current count is : 3

Increment



4. Create a form with input fields and a submit button
 - I. Create a new component named **Form**.
 - II. Use controlled components to manage input values.
 - III. Handle form submission using the **onSubmit** event handler.
 - IV. When submitting data display success message in console.



Form.jsx

```
import { useState } from "react";

function Form() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    password: "",
  });

  const handleChange = (event) => {
    const { name, value } = event.target;
    setFormData({
      ...formData, [name]: value,
    });
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    console.log("submiton successfull");
    console.log(formData);
  }

  return (
```

```

    <>
      <div>
        <form onSubmit={handleSubmit}>
          <div>
            <label htmlFor="name">Name: </label>
            <input type="text" name="name" value={formData.name}
onChange={handleChange} required/>
          </div>
          <div>
            <label htmlFor="email">Email: </label>
            <input type="email" name="email" value={formData.email}
onChange={handleChange} required />
          </div>
          <div>
            <label htmlFor="password">Password: </label>
            <input type="password" name="password" value={formData.password}
onChange={handleChange} required />
          </div>
          <button type="submit">Submit</button>
        </form>
      </div>
    </>
  );
}

export default Form;

```

App.jsx

```

import Form from './Form'
import './App.css'

function App() {
  return (
    <>
      <Form />
    </>
  )
}

export default App

```

main.jsx

```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
)

```

</StrictMode>,

A screenshot of a web browser displaying a login form and a console log. The browser has four tabs, all titled "Vite + React", and the address bar shows "localhost:5176". The login form contains three input fields: "Name" with the value "Kaihara", "Email" with the value "kaiharap84@gmail.com", and "Password" with masked characters. A "Submit" button is located below the password field. The browser's developer console is open, showing a log entry: "submiton successfull" with a link to "From, jsx:20". Below this, a JSON object is displayed:

```
{name: 'Kaihara', email: 'kaiharap84@gmail.com', password: '1234'}
```

. The Windows taskbar at the bottom shows the time as 12:35 AM and the language as ENG.

Discussion:

- In this lab practical session, we explored the fundamentals of React components, which are the building blocks of any React application. The session began with understanding what components are and how they work, including the difference between functional and class-based components. We practiced creating components and using them to build a modular UI. A key focus was on passing data between components using props, which enabled communication between parent and child components. Additionally, we learned how to manage and update the component's state, ensuring that the UI dynamically reflected changes. Finally, by building a simple form, we applied these concepts to create an interactive and responsive interface, solidifying our understanding of how React components interact and re-render based on user input. This hands-on approach helped us gain practical knowledge of structuring and managing React applications effectively.